

Mid-Term Report

高振明/Zhenming Gao

January 5, 2026

1 Current Status

As previously mentioned (2025/10/15) in my opening report, project-related coding is almost completed though there are certain aspects of *Tigris*¹ that could use a little bit more optional polishing. Specifically, we have completed

- the coding of *Tigris*' both as a language and a prover.
- passed the major modular feature test at `examples/`.

What's only left is

- the thesis.

Below, we present (non-definitive) **language specifications** and brief **internal design notes** of the language as **proof-of-progress**, which, shall include the following (based on that of OCaml):

- PL side (emphasized, in Lean):
 1. *Tigris*: The Programming language
 - (a) Lexical conventions,
 - (b) Values,
 - (c) Names,
 - (d) Type expressions,
 - (e) Patterns,
 - (f) Expressions, Declarations and Bindings
 - (g) Type(class) definitions,
 - (h) Implementation details i.e. Lexing, Parsing.
 2. Type System
 - (a) Core calculus
 - (b) Typing discipline
 - (c) Exhaustiveness checking
 - (d) Typeclass instance resolution
 - (e) System F IR & elaboration
 3. The *Lambda* IR
 - (a) Lowering
 - (b) Optimizations

¹Repository at <https://github.com/notch1p/tigris>

- (c) Incremental APIs
- 4. Codegen
 - (a) Backend: Common Lisp (SBCL)
 - i. Memory Representation
 - ii. Runtime
- 5. FFI Utilities (SBCL)
- 6. CLI Interfaces (Interpreter, Compilers)
 - (a) Tigrisi: IR-based REPL
 - (b) Tigrisl: Compiler
 - (c) Tigris: (Legacy, deprecated) Source-based REPL
- 7. Misc: Dependently-typed table printer
- Prover side (in Haskell):
 - 1. *Tigrisd*: The language itself
 - (a) Lexical conventions,
 - (b) Values,
 - (c) Names,
 - (d) Type expressions,
 - (e) Constants,
 - (f) Patterns,
 - (g) Expressions,
 - (h) ...
 - 2. Type System
 - (a) ...

Nomenclature In short: $TinyTheorem = Tigris + Tigrisd$

This document, while being a progress report only, is also an incomplete draft of the thesis' core.

Symbols	Keywords									
→ ;; ⇒	mutual	infixl	infixr	match	then	let	fun			
, _ : ∀	extern	class	forall	data	prefix	postfix	in			
	type	with	instance	else	and	rec	if			

Table 1: Reserved symbols and keywords

2 The *Tigris* language

Tigris is a purely functional, flexible, static typed language with a focus on readability and syntactic uniformity that takes inspiration from Lean, OCaml and Haskell.

2.1 Lexical conventions

Below we use an augmented form of BNF (not to be confused with ABNF) that accepts additional POSIX regex (specifically, `+[^ ]`) on the RHS.

Encoding Tigris sources are expected to be UTF-8 encoded text.

Blanks Tabs, spaces, CR/LFs are considered blanks. Blanks are ignored and thus do not affect the semantic of the program.² Blanks separate adjacent identifiers, literals and keywords in order to prevent being parsed as one single identifier.

Comments One of `--` (Lean/Haskell/Lua style), `NB.` (J style) starts a *single-line* comment. Comments are treated as if they are blanks. Note that currently multiline comments are not supported.

Identifier Almost the same as OCaml/Haskell/Lean.

```

<ident>      ::= (<letter> | _) <rest>*
<upper-ident> ::= <uppercase> <rest>*
<lower-ident> ::= <lowercase> <rest>*
<letter>     ::= <upper-letter> | <lower-letter>
<rest>       ::= <letter> | 0..9 | _ | ' | ! | ?
<lower-letter> ::= a..z
<upper-letter> ::= A..Z

```

To simplify parsing, whether an identifier is capitalized or not carries additional information. In contrast, no distinction is made between `!?`. This is permitted to facilitate the writing of idiomatic codes such as ending a nullable (**Option** `a`) or a non-local exiting function identifier in `?` or `!`, respectively.

Integer literals `<intlit>` ::= `("-" | "+")? 0..9+`, in decimal.

String literals `<strlit>` ::= `"[^"]+"`

Reserved symbols and keywords See [Table 1](#). Lexical ambiguities are resolved according to the *longest-match* rule.

²This is not the case on the prover side.

2.2 Value

Value refers to an expression that cannot be further reduced, a *normal form* so to speak. I.e. $\langle v, \sigma \rangle \Downarrow \langle v \rangle$ where v is one of

Integer numbers The range of an integer is implementation-dependent, that is, as of now, both SBCL backend and the interpreter (in Lean) support arbitrary-precision arithmetic.

Strings String values are finite sequences of characters, the size of which is implementation-dependent, that is, as of now, SBCL backend supports strings containing up to 2^{45} chars.

Tuples n -tuple is written $(v_1, \dots, v_n)$.³

Records Records are syntactic sugars of inductive types, written $\{field_1 = v_1, \dots, field_n = v_n\}$.⁴ Currently, no type-directed parsing is implemented because of potential major refactors. Thus for records with same field labels, regardless of types, an ambiguous parsing warning is thrown but can be avoided with type ascription (same with patterns):⁵

```
type Point = {x : Int, y : Int}
type Point' = {x : Int, y : Int}

-- [WARN] ambiguous record literal (using default 'Point'), consider adding type ascription;
-- as we currently do not have type-directed parsing.
-- candidates are [(Point, #[x, y]), (Point', #[x, y])]
let _ = {x = 1, y = 2}

let _ = ({x = 1, y = 2} : Point') -- success
```

For overlapping labels, a *longest-match* rule is used:

```
type Point = {x : Int, y : Int}
type Point3D = {x : Int, y : Int, z : Int}
let (x, y) = ( {x = 1, y = 2} -- x : Point
              , {x = 1, y = 2, z = 3} ) -- y : Point3D
```

Constructed values refer to values that are constructed by a *value constructor* (or simply unqualified, *constructor*) of some inductive type. It is also called a *variant*. Constructors are like functions except they may not have parameters: for some constructor C_i , a variant is either C_i or $C_i \text{ fid}_1 \dots \text{fid}_n$. The following variants are built-in:

Constructors	Types
true	Bool
false	Bool
()	Unit

Functions refers to the mappings from values to values. Details explained later.

2.3 Names

Identifiers are used to reference *language objects* by name: [Table 2](#)

³Tuples are specially treated instead of the usual `Prod` encoding.

⁴Typeclass instances are syntactic sugars of records.

⁵This annotation must be “next to” the record literal as it is processed by the parser rather than the typechecker.

Category	First letter
Values	any
Constructors	uppercase
Type constructors	uppercase
Record fields	any
Bound type variables	lowercase

Table 2: Case difference of identifier categories

Operator	Associativity
Type application	left
Product type (*)	right
Arrow type ($\rightarrow$)	right

Table 3: type operator associativity

2.4 Type expressions

<code><type-expr></code>	<code>::= <type-app> (-> <type-expr>)?</code>	(arrow type)
<code><type-atom></code>	<code>::= <ident></code>	
	<code> "(" <type-expr> ")"</code>	(parenthesized)
	<code> <type-ctor></code>	
	<code> sepBy(* ×, <type-expr>)</code>	(product type)
	<code> <type-args></code>	
<code><type-app></code>	<code>::= <type-app>? <type-atom></code>	(type application)
<code><type-ctor></code>	<code>::= <upper-ident></code>	
<code><bound-ty></code>	<code>::= <lower-ident></code>	
<code><type-binder></code>	<code>::= <bound-ty></code>	(type binder)
	<code> "(" <bound-ty> : <intlitt> ")"</code>	
<code><type-args></code>	<code>::= <bound-ty> <type-expr></code>	(type argument)
<code><qualified></code>	<code>::= "[" sepBy1(",", (<bound-ty> <type-ctor>) <type-expr>+) "]"</code>	
<code><scheme></code>	<code>::= { FORALL <type-binder>+ }? <qualified>? ", " <type-expr></code>	
FORALL	<code>::= forall ∀</code>	

Note that, for juxtapositional type application⁶, product type and arrow type, we have the following precedence & associativity table (higher comes first): [Table 3](#)

Type variables type variables (TVs) are identifiers with a lowercase beginning. The type (kind) of a TV is determined explicitly to aide the absent of kind inference. A TV denoted $(\alpha : n)$ is $\text{Type} \rightarrow \text{Type}$ which otherwise is Type .⁷ In datatype definitions, type variables are always bounded, appearing at parameter position as well a subsequent type expression on the RHS. Note that validity checking of type expressions are mostly done at parsing time i.e. If some TV is unbound, a parsing error is immediately thrown. In type constraints a TV may be one of:

⁶similar to the a/b-type implementation in Haskell 2010 report.

⁷Note that however we do not allow curried HKT i.e. $(\alpha : 2)$ is instead, $\text{Type} \times \text{Type} \rightarrow \text{Type}$. Because of this, we shall refer to it as *arity* rather than *kind* from now on

metavariables Also called *MVs*, denoted $?m.N$ where N is a counter. Refers to a unspecified type that *must* be either instantiated⁸ by a type or generalized when leaving the scope⁹ MVs must not appear in the final scheme.

skolem-TVs Also called *skolems*, denoted $?sk.A$ where A is the original TV. Skolem variables are TVs produced by the process of skolemization. These TVs are essentially type constructors¹⁰ that may *not* be instantiated.

Note that, there is currently no implicit inference of type variables. They can only be introduced at a $\forall$ binder or datatype declarations.

Function types The type expression $ty_1 \rightarrow \dots \rightarrow ty_n$ denotes the type of functions mapping of $ty_1, \dots, ty_{n-1}$ to results of type ty_n .

Product types The type expression $ty_1 \times \dots \times ty_n$ denotes the type of tuples whose elements are of $ty_1 \times \dots \times ty_n$ respectively.

Constructed types Similar to TVs, type constructor has arity as well. $<type-ctor>$ is a type constructor with no parameter and a type expression. Otherwise, a type constructor must be fully applied.

arbitrary recursive types is rejected thus it is not possible to type `fun x -> x x`.

2.5 Patterns

Patterns, as in pattern matching, are templates of instructions used to *destructure* data structures of some shape. Patterns can be enriched by custom operators (described later) if they are associated with constructors however they are reduced to one of, or the composition of the primitive patterns below at parsing-time:

<code><pat></code>	<code>::= <pat-left> <infix-op> <pat></code>	infix operator enriched
	<code><pat-left></code>	
	<code><prefix-op> <pat></code>	prefix operator enriched
	<code><pat> <postfix-op></code>	postfix operator enriched
<code><pat-left></code>	<code>::= <pat-atom></code>	
	<code><ctor> many1(<pat-atom>)</code>	variant
<code><pat-atom></code>	<code>::= <lower-ident></code>	variable
	<code><ctor></code>	constructor
	<code>"_"</code>	wildcard
	<code>"{" sepBy1(",", <pat-field>) "}"</code>	record pattern
	<code>"(" sepBy1(",", <pat>) ")"</code>	product pattern
	<code>"(" <pat> : <type-expr> ")"</code>	ascribed pattern
	<code><literal></code>	
<code><pat-field></code>	<code>::= <field> = <pat></code>	

Note that the arity of a constructor must match the number of sub-patterns associated with it i.e. constructor in this context must not be partially applied. Mixfix (i.e. {pre,in,post}-fix) operators have the following precedence in patterns (higher comes first): [Table 4](#)

⁸The word *instantiate* is used throughout this text while its usage is not all well-defined: Sometimes, instantiate means *specialize*, where TVs are instantiate to concrete types; sometimes, it only indicates the substitution of a TV.

⁹that is, the whole toplevel phrase.

¹⁰that is, encoded as TCons to prevent illicit unification

Operator	Associativity
Constructor application	left
Prefix operator	
Postfix operator	
Infix operator	left
Infix operator	right

Table 4: Precedence of operators in patterns

variable & wildcard patterns binds the name to the value, e.g.:

```
let fst (x, _) = x
```

Unlike other languages, pattern are not linear. A variable can be bound several times by a given pattern and overwrites previous bindings. Beware of the semantic confusion:

```
let prodEq : ∀a b, a × b → Bool
  | (x, x) ⇒ true
  | (x, _) ⇒ false
-- [WARN] Found redundant alternatives at
-- 2) #[x, _]
```

It is not possible to decide the equality relation between two parts of a data structure through arbitrary patterns. Use the method (=) of typeclass Eq for that matter.

literal patterns test the equality of the constants.

```
let strToBool : String → Bool = fun
  | "true" ⇒ true
  | "false" ⇒ false
-- [WARN] Partial pattern matching, an unmatched candidate is
-- [_]
```

parenthesized patterns come into effect when mixing with operator-enriched patterns:

```
type List a = Nil | Cons a (List a)
infixr 90 "::" ⇒ Cons
```

```
let f1 (x :: y :: _) = (x, y)
let f2 ((x :: y) :: _) = (x, y)
```

f1 and f2 are vastly different, according to *Tigris*' System F IR:

```
let (::) : ∀ α, α → List α → List α = (λ α. Cons@α)

let f1 : ∀ α, List α → α × α =
  (λ α. fun ?x₀ : List α ⇒ match ?x₀ with | Cons x (Cons y _) ⇒ (x, y))

let f2 : ∀ α, List (List α) → α × List α =
  (λ α. fun ?x₀ : List (List α) ⇒ match ?x₀ with | Cons (Cons x y) _ ⇒ (x, y))
```

variant patterns are of the form $C_i p_1, \dots, p_n$. It matches all variants whose constructor equals to C_i and whose subpatterns matches $p_1, \dots, p_n$. An arity mismatch error is signaled if the number of subpatterns is unexpected by the constructor.

Record patterns are similar to record literals that will encounter ambiguous parsing if there exists records with same field labels.

```

type Point = {x : Int, y : Int}
type Point' = {x : Int, y : Int}

let sum {x, y} = x + y
-- [WARN] ambiguous record pattern (using default 'Point'), consider adding type ascription;
-- as we currently do not have type-directed parsing.
-- candidates are [(Point, #[x, y]), (Point', #[x, y])]
let sum' ({x, y} : Point') = x + y

```

Since records are syntactic sugar of inductive types, same applies to record patterns. The denotational semantic is

$$\llbracket \text{type } T \text{ args}^* = \{(fid : \sigma)^*\} \rrbracket := \text{type } T \text{ args}^* = T \sigma^*$$

where tm^* denotes the *splice* of tm , a sequential structure. Thus, a field can be extracted using the ordinary constructor application pattern:

```

class Functor (m : 1) =
  {map : forall a b, (a → b) → m a → m b}
let getMap (Functor f) = f

```

yields IR

```

let getMap : ∀ α, Functor α → ∀ a b, (a → b) → α a → α b =
  (λ α. fun ?x₀ : Functor α ⇒ match ?x₀ with | Functor f ⇒ f)

```

2.6 Expressions

Expressions are the core of *Tigris*. Colloquially, we define the following metavariables:

- p_i denotes the i -th pattern in a pattern matching branch.
- C_i denotes the i -th constructor in a inductive type definition
- e, e', e_i denotes an arbitrary expression whose evaluation value depends on the context.
- v, v', v_i denotes a irreducible value.
- $a, b, c, d, e, x, y, z, w$ denotes a binder, or an identifier.
- f, g, h denotes an identifier and semantic-wise, a function.
- $\llbracket \rrbracket$ denotes blackboard style brackets, sometimes used as delimiters (if usable ordinary delimiters have run out) in metalanguage.
- greek letters mostly denotes types. Greek letters with a bar on them $\bar{\alpha}$ denotes a list of those letters, with implicit numbering $1, \dots, i$.

A *Tigris* expression will, eventually, after exhaustive application of transformations, be translated into the core calculus **Expr** defined in `types.lean`.

<code><expr></code>	<code>::= <expr-infix></code>	
	<code> (<expr-infix> : <type-expr>)</code>	ascription
<code><expr-infix></code>	<code>::= <expr-left> <infix-op> <expr-infix></code>	infix operator
	<code> <prefix-op> <expr-left></code>	prefix operator
	<code> <postfix-op> <expr-left></code>	postfix operator
	<code> <expr-left></code>	
<code><expr-left></code>	<code>::= fun (<expr-binder>+ ARROW <expr> <expr-match>)</code>	lambda abstraction
	<code> let "rec"? <pat'> = expr in <expr></code>	pattern matching let
	<code> let "rec"? <let-binder> sepBy("and", <let-binder>)</code>	let(rec) expression
	<code>in <expr></code>	
	<code> if <expr> then <expr> else <expr></code>	conditional
	<code> rec <expr></code>	fixpoint; deprecated
	<code> match sepBy1(",", <expr>) with <expr-match></code>	pattern matching
	<code> <expr-funapp></code>	
<code><expr-funapp></code>	<code>::= <expr-funapp>? <expr-atom></code>	function application
<code><expr-atom></code>	<code>::= <literal></code>	
	<code> <expr-ctor></code>	
	<code> <lower-ident></code>	variables
	<code> "(" sepByN(",", <expr>) ")"</code>	unit
		$N = 0$
		$N = 1$
		$N \geq 2$
	<code> "{" sepBy1(",", <expr-field>) "}"</code>	product
<code><expr-ctor></code>	<code>::= <upper-ident></code>	record literal
<code><expr-match></code>	<code>::= (" " sepBy1(",", <pat>) ARROW <expr>)+</code>	constant constructors
<code><expr-field></code>	<code>::= <field> = <expr></code>	pattern match body
<code><expr-binder></code>	<code>::= <id> <pat'></code>	binders
<code><pat'></code>	<code>::= "(" <pat> ")"</code>	parenthesized <pat>
	<code> <literal></code>	except literal patterns
<code><let-binder></code>	<code>::= <pat'> = <expr></code>	single-pattern binding
	<code> <ident> <expr-binder>* <type-expr>? <let-rhs></code>	value/function binding
<code><let-rhs></code>	<code>::= "=" <expr> <expr-match></code>	let binding RHS
<code><literal></code>	<code>::= <intlit> <strlit> "true" "false"</code>	
<code>ARROW</code>	<code>::= "⇒" "→"</code>	

Note that, the ARROW lexer parses both lexeme $\Rightarrow$ and $\rightarrow$, only the first should be used, the alternative is to add some form of ML compatibility.

Basic expressions are all expressions list in `<expr-atom>` with the addition of function shorthand syntax.

Shorthand syntax is a Lean-like syntactic sugar for parenthesized expressions, within which, an underscore (`_`) or `cdot` (`·`) is transformed in preorder¹¹ against the expression to a function binder. This is similar to Haskell's operator sectioning. Respects precedence, e.g.

¹¹i.e. DFS.

	<code>fun x $\Rightarrow$ x + 1</code>
<code>(_ + 1)</code>	<code>fun x y $\Rightarrow$</code>
<code>(_ + 1 * 2 / _)</code>	<code> let tmp = 2 / y in</code>
	<code> let tmp = 1 * tmp in</code>
	<code> x * tmp</code>
<code>(id _)</code>	<code>fun x $\Rightarrow$ (id : forall a, a -> a) x</code>
(a) shorthand syntax	(b) desugared results

Parenthesized expressions can also contain type constraint, as in the example above.

Curried applications & definition Functional values are curried.¹² i.e. partial application is allowed and is in practice frequently used in idiomatic functional programming.

Function application associates to the left. It is denoted by juxtaposition of expressions. *Tigris* is call-by-value (CBV), thus the expression $e\ e'$ evaluates e to a functional value first, then e' .

Lambda abstraction takes the form of `fun  $p_1 \cdots p_n \Rightarrow e$`

Function definition has two syntactic forms:

$$\begin{array}{l}
 \text{fun} \\
 | \ p_{1,1}, \dots, p_{1,m} \Rightarrow e_1 \\
 \vdots \\
 | \ p_{n,1}, \dots, p_{n,m} \Rightarrow e_n
 \end{array} \tag{1}$$

$$\text{fun } p_1 \cdots p_m \Rightarrow e \tag{2}$$

Both forms are syntactic sugar that are translated and, equivalent to

$$\begin{array}{l}
 \text{fun } x_1 \Rightarrow \cdots \text{fun } x_n \Rightarrow \\
 \text{match } x_1, \dots, x_n \text{ with} \\
 | \ p_{1,1}, \dots, p_{1,m} \Rightarrow e_1 \\
 \quad \text{applies to form (1)} \\
 | \ \overbrace{p_{2,1}, \dots, p_{2,m}} \Rightarrow e_2 \\
 \vdots \\
 | \ p_{n,1}, \dots, p_{n,m} \Rightarrow e_n
 \end{array}$$

Syntactic intuition shows that canonically, a lambda abstraction only takes one (1) parameter.

Let-bindings & expressions The `let` and `let rec` language constructs bind value names either locally or globally, together with `in <expr-body>`, they form expressions.

¹²Constructors and functions form an *exhaustive partition* of functional value.

Nomenclature Here, a *let-binding* describes the $p_n = e_n$ portion, and-branches are implicitly assumed until mentioned otherwise; A *let-expression* includes the *body* portion as well.

Local non-recursive definitions the construct **let** $p_1 = e_1$ **and** $\dots$ **and** $p_n = e_n$ **in** e evaluates¹³ $e_1 \dots e_n$ and matches the results against pattern $p_1 \dots p_n$. If matching succeeds, e is evaluated in the lexical scope enriched by the bindings produced during matching, whose value is returned as the value of the whole let expression.¹⁴ Pattern matching let-expression is translated to a **match** .. **with** construct.

The canonical let-binding described above can be used to bind functional values though syntactic sugar exists: In a let expression, function may be defined as **let** $f\ p_1 \dots p_n = e$, e.g.

```
let prodSum (x, y) z w = x + y + z + w
and x = 1
and y = 2
in prodSum (x, y) 3 4

let (x, y) = (1, 2) in x + y

let 0 = 1

let compose f g x = f (g x) in
let add20 = (_ + 20) in
let minus30 = fun x => x - 30 in
compose add20 minus30 50
```

In the third example, a warning is thrown;

```
[WARN] Partial pattern matching, possible cases such as [true] are ignored
```

At runtime, the unrecoverable exception MATCH-FAILURE is thrown:

```
debugger invoked on a MATCH-FAILURE in thread
#<THREAD tid=145807 "main thread" RUNNING {1200030003}>:
  No branch can be matched against #[Toplevel]
```

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):

```
0: [ABORT] Exit debugger, returning to top level.
```

```
(|__start|)
  source: (FUNCALL #'|main| NIL #'IDENTITY)
0]
```

Local recursive definitions are introduced by the **let rec** construct:

$$\text{let rec } f_1\ p_1 \dots p_n = e_1 \text{ and } \dots \text{ and } g \dots = e_n \text{ in } e$$

To grossly simplify the feature of **let rec**: the names are considered already bound when the expressions e_1 to e_n are evaluated. In practice, the importance of this construct is obvious as this is the only way to facilitate (mutual) recursion in *Tigris*.

¹³virtually (in the context of a typechecker), in parallel; In practice, this is implementation-dependent. Cf. nested let.

¹⁴no sequential binding is performed. A name bounded to e_i is unbounded in e_{i+1} .

```

mutual
type Tree a = Empty | Node a (Forest a)
type Forest a = Nil | Cons (Tree a) (Forest a)
;;

let rec countTree
  | Empty ⇒ 0
  | Node _ f ⇒ 1 + countForest f
and countForest
  | Nil ⇒ 0
  | Cons t f ⇒ countTree t + countForest f

```

New users may be tempting to define recursive functions in a more “functional” way, but hesitate as they do not know whether it will work: **let rec** $f = \text{fun} \dots \text{and} \dots \text{and } g = \text{fun} \dots \text{in } e$. The answer is yes: Just the same as **let** binding, the canonical form of a **letrec** expression is precisely the above, what we introduced earlier is simply a syntactic sugar. It defines f , g as mutually recursive functions local to e . This nature is reflected in the core calculus as whether recursive or not a **let rec** binding is, it is always transformed into a **Let** node, with the only difference being the RHS of a **let rec** binding is wrapped in **Fix**.

The semantic difference between **let rec** and **let** is minute, yet complicated. We largely follow OCaml’s implementation but is even simpler and more strict.

Remark (Partition heuristic). the RHS of a **letrec** binding must have the form of **fun** ..; Otherwise, it is ruled as a **let** binding.

Using a simple¹⁵ heuristic,¹⁶ we partition a **letrec** group into one that is strictly *recursive*¹⁷ and another one that becomes regular *let-binding*. Additionally, we require the LHS of a **letrec** binding to always be variables.¹⁸ These assumptions are strict enough to rule out some valid programs.

```
let rec x = (1, x) and y = (2, x)
```

```
let rec x = (1, y) and y = 1
```

The first example is invalid in CBV semantics as it creates cyclic products whereas the second one is ruled out though valid. We find it appropriate as though example 2 is invented, it’s coding style is inherently bad; Besides, the author also struggles to find a convincing example that justifies the preservation of the expressiveness lost here.

Global definitions are just like local ones, but is limited to the toplevel, where a **let(rec)** expression can be free of its body i.e. becomes a standalone **let** binding. Multiple toplevel **let** bindings can optionally be separated with **;;** for clarity.

Control structures

Sequential operations are not builtin, one may substitute it with nested **let** expressions.

Conditional expressions are of the form **if** e **then** e_1 **else** e_2 ; which evaluates to e_1 if e yields the bool constructor **true** otherwise e_2 .

¹⁵no additional free occurrence checking, or much of the condition checking in the OCaml manual ch. 12.1.

¹⁶one of the condition to be *statically constructive*.

¹⁷in the sense there is no recursive definition of non-functional values.

¹⁸Unlike Haskell, which translate it into a fixpoint application with pattern matching.

Case analysis Also called pattern matching. The expression

```

match  $e_1, \dots, e_m$  with
  |  $p_{1,1}, \dots, p_{1,m} \Rightarrow e_1$ 
  |  $p_{2,1}, \dots, p_{2,m} \Rightarrow e_2$ 
  :
  |  $p_{n,1}, \dots, p_{n,m} \Rightarrow e_n$ 

```

matches the row expression vector against the pattern matrix *in parallel*,¹⁹ yielding the RHS of a row that first succeeds. At compile time, pattern matrices are checked of exhaustiveness and redundancy, using an efficient matrix-modeled algorithm from Maranget:

```

let (1, 2) = (1, 2) -- [WARN] Partial pattern matching, possible cases such as [(_, _)] are ignored

let f
  | (1, 2)  $\Rightarrow$  1 + 2 -- [WARN] Found redundant alternatives at
                    -- 3) #[(_, y)]
  | (x, _)  $\Rightarrow$  x
  | (_, y)  $\Rightarrow$  y

```

If none matches, the nonrecoverable runtime exception MATCH-FAILURE is thrown. Pattern matching can be seen as a superset of the switch construct found elsewhere. It enables a declarative paradigm to elegantly destructure data structures and is at the core of modern functional programming.

Operators Operators are special strings bound to expressions. They are recognized by the parser and can be chained in expressions.

One feature that set *Tigris* apart from other languages is its ability to define custom operators from a infinite set of strings with associativity and precedence adjustable. A custom operator definition is either a 4-tuple ($op, prec, assoc, impl$) or a 3-tuple ($op, prec, impl$), collected from one of the three operator declaration forms below:

```

<infixl-decl> ::= infixl ":"? <op> <prec> ARROW <expr>      ;  $\Rightarrow$  (<op>, <prec>, L, <expr>)
<infixr-decl> ::= infixr ":"? <op> <prec> ARROW <expr>      ;  $\Rightarrow$  (<op>, <prec>, R, <expr>)
<prefix-decl> ::= prefix ":"? <op> <prec> ARROW <expr>      ;  $\Rightarrow$  (<op>, <prec>, <expr>)
<postfix-decl> ::= postfix ":"? <op> <prec> ARROW <expr>     ;  $\Rightarrow$  (<op>, <prec>, <expr>)

```

Where <op> denotes the operator string and <op> itself is a subset of <strlit>.

Definition (Operator candidate strings). Unlike Haskell or OCaml, whose <op> comes from combinations of a predefined set of operator characters, *Tigris* follows a Lean-like approach and defines <op> to be *any string except*:

- those that begin with numbers,
- those that begin with or contain one of `[_ , ( [ { } ] ) ]`,
- those that, except the beginning, contain <blank>,
- those that equal to a reserved operator shown here: [Table 1](#).

¹⁹Cf. product pattern, which is evaluated in preorder.

Operator	Precedence	Class	Implementation
*	70	infixl	Primitive integer multiplication
/	70	infixl	Primitive integer division
+	65	infixl	Primitive integer addition
-	65	infixl	Primitive integer subtraction
=	50	infixl	Primitive integer equality
\$	1	infixr	Function application
@@	1	infixr	Function application

Table 5: Predefined operators

Blank characters *surrounding* <op> are automatically trimmed.

Precedence of the four operator classes (infixl, infixr, prefix, postfix) with respect to function application is the same as in patterns: Table 4. Although much²⁰ of the operators are left for users to define, some basic operators are defined (though they can be overridden) using the same mechanics.^{21 22} The addition of \$ and @@ is a clever abuse of precedence and is to facilitate idiomatic function programming techniques. With the lowest precedence, it is possible to avoid parenthesization nested to the right, as in:

```
let _ = f (g (h (x + y)))
let _ = f $ g $ h $ x + y
let _ = f @@ g @@ h @@ x + y
```

This becomes particularly handy if the argument of some function is a large expression and this technique is also widely used throughout the implementation of *Tigris* and *Tigris*d themselves. It is also possible to bound a same operator to both prefix and postfix:

```
let not
  | true ⇒ false
  | false ⇒ true
let rec fact
  | 0 ⇒ 1
  | n ⇒ n * fact (n - 1)
```

```
postfix 65 "!" ⇒ fact
prefix 65 "!" ⇒ not
let main = (50!, !(true))
```

But currently it is not possible to do so between prefix and infix or postfix and infix. Common idiomatic use includes defining the Cons constructor of List as (::) and append as (++):

```
type List a = Nil | Cons a (List a)
infixr 67 " :: " ⇒ Cons
let car (x :: _) = x
```

²⁰that is, an (almost) infinite amount of potential operators.

²¹Though in practice it is more common to bind operators with methods from typeclass, we chose to keep it simple; Besides, It is also true that we currently do not have a *prelude*.

²²As described in the expression grammar, following the usual practice, language constructs are hardcoded to have lower precedence than operators, thus `let n = 10 in n + x` parses as `let n = 10 in (n + x)` rather than `(let n = 10 in n) + x`.

```

let cdr (_ :: xs) = xs
let rec map f
  | Nil => Nil
  | x :: xs => f x :: map f xs
let foldl1 f xs =
  let rec go xs =
    match xs with
    | x :: Nil => x
    | x :: y :: xs =>
      go (f x y :: xs)
  in go xs

infixl 65 " ++ " => append
let rec append
  | Nil, ys => ys
  | x :: xs, ys => x :: append xs ys

```

Multiple toplevel operator bindings can optionally be separated with `;;` for clarity.

2.7 Type, typeclass and instance definitions

datatype definition is used to introduce new types to the environment. It binds type constructors to datatypes in either of the forms: inductive types or record types. Though for the latter it is merely a syntactic sugar of the former as mentioned before.

```

<type-decl> ::= mutual <type-decl1>+ ";;" (mutual rectype definition)
              | <type-decl1>
<type-decl1> ::= TYPE <type-ctor> <type-binder> = <type-constrs>
<type-field> ::= <ident> : <type-expr>
<type-constrs> ::= <type-record>
                  | ("|" <type-ind>)+
<type-record> ::= "{" sepBy1(",", <ident> : <type-expr>) "}"
<type-ind> ::= <expr-ctor> <type-atom>+ (inductive type branch)
TYPE ::= type | data

```

Note that, the TYPE lexer parses both lexeme type and data, only the first should be used, the alternative is to add some form of Haskell compatibility.

An inductive type has the form

$$\mathbf{type} \, T \, \bar{\alpha} = C_1 \sigma_{1,1} \cdots \sigma_{1,c_1} \mid \cdots \mid C_n \sigma_{n,1} \cdots \sigma_{n,c_n}.$$

This declaration introduces a new type constructor T that takes n parameter *fully*, with one or more value constructors $C_1, \dots, C_n$. The types of the constructors are given by

$$C_i : \sigma_{i,1} \rightarrow \cdots \rightarrow \sigma_{i,c_i} \rightarrow T \, \bar{\alpha}.$$

where the lowercase c_i represents the arity of C_i .²³

²³The result type of a constructor must match the inductive type declaration as indexed families are not currently supported.

```

type RGB = {r : Int, g : Int, b : Int}
type Color =
  | Red | Green | Blue | Yellow | Black | White
  | Custom RGB

type Tree a = Lf | Br a (Tree a) a

mutual
type Tree a = Empty | Node a (Forest a)
type Forest a = Nil | Cons (Tree a) (Forest a)
;;

```

Type synonyms (equality constraints) and *contexts* (predicates) are currently not supported for type declarations. Note that, *Tigris* currently does enforce strict positiveness in inductive types.

Definition (Strict Positiveness). A position is said to be strictly positive if

1. it is not in a function's argument type, and,
2. it is not an argument of any expression other than type constructors of inductive types.

Since *Tigris* is a programming language, we do not make this restriction as it may rule out some unproblematic type definition.

Typeclass definition Typeclass is the functional counterpart to what is called an *interface* in Object-oriented programming, except, it is more efficient and well-defined, as in methods are dispatched statically, as in enabling ad-hoc/parametric polymorphism in a principled fashion.

```

<class-decl> ::= class <class-ident> <type-binder> = <type-record>
<class-ident> ::= <type-ctor>

```

A class declaration introduces new class methods, whom are globally scoped. Classes are very similar to records because of their implementation²⁴, except it utilizes the typeclass facility. A class declaration takes the form of

$$\text{class } C \ \bar{\alpha} = \langle \text{type-record} \rangle$$

and the type of the class method f_i is

$$f_i : \forall \bar{\alpha}, \bar{\beta}. \sigma_i$$

where $\bar{\beta}$ is the additional type variables bound at method level²⁵, and σ_i is the type signature of f_i modulo binder. Consider

```

class Eq a =
  {eq : a -> a -> Bool}
class Functor (f : 1) =
  {map : forall a b, (a -> b) -> f a -> f b}
class HEq a b =
  {heq : a -> b -> Bool}

let main = (map, eq, heq)

```

²⁴dictionary passing.

²⁵supported by polymorphic fields.

eq has type $\forall a [\text{Eq } a]. a \rightarrow a \rightarrow \text{Bool}$, map has type $\forall f [\text{Functor } f]. \forall a b. (a \rightarrow b) \rightarrow fa \rightarrow fb$, heq has type $\forall a b [\text{HEq } a b]. a \rightarrow b \rightarrow \text{Bool}$; Whereas main, i.e. some program that uses all the methods above without providing additional type ascription to *refine* the class, yields the verbose

$$\forall \alpha \beta \gamma \delta [\text{Functor } \alpha, \text{Eq } \beta, \text{HEq } \gamma \delta] \forall a b. ((a \rightarrow b) \rightarrow \alpha a \rightarrow \alpha b) \times (\beta \rightarrow \beta \rightarrow \text{Bool}) \times (\gamma \rightarrow \delta \rightarrow \text{Bool}).$$

Default methods and superclass are not supported at this moment.

Instance definition Just as in object-oriented programming which interfaces are always associated with their implementations, so is typeclass. *Instances* are used to implement typeclasses.

```
<inst-decl> ::= instance ":"? <inst-scheme> = "{" <inst-method-decl> "}"
<inst-scheme> ::= {FORALL <type-binder>+}? <inst-context>? ", " <class-ident> <type-expr>+
<inst-context> ::= "[" sepBy1(", ", <qualified>) "]"
<inst-method-decl> ::= sepBy(", ", <let-binder>)
```

For the sake of syntactic uniformity, the RHS of a instance declaration is similar to a record literal, with its field assignments being similar to a let-binding.

```
instance : Eq Int = {eq x y = __eqInt x y} -- __eqInt is primitive, requires a-conversion
instance : Functor Option =
  { map f
    | Some x => Some $ f x
    | None   => None
  }
instance HEq String Bool =
  { heq = fun
    | "true", true   => true
    | "false", false => true
    | -, -          => false
  }
instance forall a b [HEq a b], HEq b a = {heq y x = heq x y}
instance forall a [Eq a], HEq a a = {heq = eq}
```

Instance declarations may quantify type variables, Together with contexts, this can be used (as shown in example 4 below) to express the *implication* relations.²⁶ Here, we quantify a, b and an instance $\text{HEq } a b$ and use it as the implementation for $\text{HEq } b a$ to express the assumed *commutative* nature of a heterogenous equality and to express the fact that the homogenous equality relation subsumes the hetero one. Instance resolution is described later. For now, users only need to remember that

1. a type may not be declared as an instance of some class more than once.
2. overlapping instance results in a compile-time ambiguous exception.
3. ambiguous type arising from the use of a method blocks typeclass elaboration and throws a compile-time ambiguous exception.

```
class HAdd a b c = {hadd : a -> b -> c}
instance HAdd Int Int Int = {hadd = add}
```

```
-- Ambiguous: HEq ?m6 Int: typeclass elaboration is stuck because of
-- metavariable(s)
```

²⁶That is, the $\rightarrow$ in first-order logic, or $\vdash$ in metalanguage.

```
-- [?m6]  
-- induced by a call to heq. To resolve this, consider adding type ascriptions.  
let f' = heq (hadd 2 3) 5
```

In this example, `hadd 2 3` is typed a metavariable, results in HEQ's instance lookup failure.

Although it is entirely possible to infer the context in a instance declaration just the same as in `main` in the above example, we nevertheless require users to provide explicit instance context.

Multiple toplevel type declarations can optionally be separated with `;;`²⁷ for clarity.

2.8 Wrapping Up: Brief implementation notes

Below we briefly talk about some key points of *Tigris*' implementation, though readers shall expect a more thorough discussion in the thesis.

Lexing & Parsing Lexing stage and Parsing stage are fused. We employ the functional way of *monadic parsing*, which typically uses *parser combinators*, together with the flexible custom operators, an implementation of a fusion of combinatory parsing and Pratt parsing is ultimately used.

In practice, parser combinators are similar to a recursive descent parser or a hand-written one. The grammar is similar to a parsing expression grammar (PEG) which always selects the first production when encountering ambiguous rules. Though it is currently, not a packrat parser which is guaranteed to run in linear time. Backtracking is still extensively used in our case as a result of the our monad class implementation. A parser monad in *Tigris* is defined to be

```
abbrev TParser  $\sigma$  := SimpleParserT Substring Char  
$ StateRefT (PEnv  $\times$  String) (ST  $\sigma$ )
```

Where σ is a formal argument for the ST-backed parser monad, allowing efficient and proper mutable state to be stored in threadlocals rather than a simulated `StateT` that uses stack space. The monad transformer stack also has two transformers `SimpleParserT` and `StateRefT` to allow the fluent use of State-like methods in `ST` as well as the efficient use of `Substring` where we do not allocate new strings and instead uses a pair of start/end pointer as a *view* during parsing.

According to Wikipedia, the two major parser combinators are (given two parsers P and Q)

- the *alternative* combinator $p \oplus q (j) = P j \cup Q j$
- the *sequence* combinator $(p \otimes q) (j) = \bigcup \{Q k \mid k \in P j\}$

This is exactly, captured by the `MonadExcept`, `Alternative` class and the `Seq` class in monadic functional programming, and is implemented as

```
1 instance ( $\sigma \varepsilon \tau m$ ) [Parser.Stream  $\sigma \tau$ ] [Parser.Error  $\varepsilon \sigma \tau$ ] [Monad m]  
2   : Monad (ParserT  $\varepsilon \sigma \tau m$ ) where  
3   -- (...)  
4   seq f x s := f s >>= fun  
5     | .ok s f  $\Rightarrow$  x () s >>= fun  
6     | .ok s x  $\Rightarrow$  return .ok s (f x)  
7     | .error s e  $\Rightarrow$  return .error s e  
8     | .error s e  $\Rightarrow$  return .error s e  
9  
10 instance ( $\sigma \varepsilon \tau m$ ) [Parser.Stream  $\sigma \tau$ ] [Parser.Error  $\varepsilon \sigma \tau$ ] [Monad m] :
```

²⁷except mutual block where `;;` is required to close it.

```

11 MonadExceptOf  $\varepsilon$  (ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$ ) where
12   throw e s := return .error s e
13   tryCatch p c s := p s  $\gg$  fun
14     | .ok s v  $\Rightarrow$  return .ok s v
15     | .error s e  $\Rightarrow$  (c e).run s
16
17 instance ( $\sigma$   $\varepsilon$   $\tau$   $m$ ) [Parser.Stream  $\sigma$   $\tau$ ] [Parser.Error  $\varepsilon$   $\sigma$   $\tau$ ] [Monad  $m$ ] :
18   OrElse (ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$   $\alpha$ ) where
19   orElse p q s :=
20     let savePos := Parser.Stream.getPosition s
21     p s  $\gg$  fun
22       | .ok s v  $\Rightarrow$  return .ok s v
23       | .error s _  $\Rightarrow$  q () (Parser.Stream.setPosition s savePos)

```

Basic scaffolding and common combinators are already done by Dorais' [lean4-parser](#) and our implementation depends on this package. Additionally, we have defined several combinators are defined to aid the parsing of function application in `utils.lean` (especially, `chainl` and `chainr`):

```

1  /--
2    `chainl p op` parses *one* or more occurrences of `p`, separated by `op`. Returns
3    a value obtained by a **left** associative application of all functions returned by `op` to the
4    values returned by `p`.
5    - if there is exactly one occurrence of `p`, `p` itself is returned touched.
6  -/
7  partial def chainl1
8    (p : ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$   $\alpha$ )
9    (op : ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$  ( $\alpha \rightarrow \alpha \rightarrow \alpha$ )) : ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$   $\alpha$  := do
10    let x  $\leftarrow$  p; rest x where
11    rest x :=
12      (do let f  $\leftarrow$  op; let y  $\leftarrow$  p; rest (f x y))  $\triangleleft$  pure x
13
14  partial def chainr1
15    (p : ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$   $\alpha$ )
16    (op : ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$  ( $\alpha \rightarrow \alpha \rightarrow \alpha$ )) : ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$   $\alpha$  := do
17    let x  $\leftarrow$  p; rest x where
18    rest x :=
19      (do let f  $\leftarrow$  op; chainr1 p op  $\triangleleft$  f x)  $\triangleleft$  pure x
20  /- like `test`, but non-consuming -/
21  def test' (p : ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$   $\alpha$ ) : ParserT  $\varepsilon$   $\sigma$   $\tau$   $m$  Bool :=
22    try lookAhead p  $\$$  true
23    catch _  $\Rightarrow$  return false

```

Typechecker There are two typecheckers available in the *Tigris* repository, a legacy, straight implementation of Algorithm W, as well as a constraint solver version that performs better with qualified types and is actually used. We shall only discuss the latter. Typecheckers are run inside the `InferC` `Monad`:

```

1  abbrev InferC  $\sigma$  := StateRefT CState (EST TypingError  $\sigma$ )
2
3  local instance : MonadLift (Except TypingError) (InferC  $\sigma$ ) where
4    monadLift

```

```

5      | .error e ⇒ throw e
6      | .ok res ⇒ return res

```

A tone-downed version of GHC’s type variable leveling is used to deal with erroneous unification. Pattern matrices are checked at this stage of exhaustiveness and redundancy, using an efficient matrix-modeled algorithm from Maranget.

The typechecker checks type, as well as producing a typed AST, which will then be *elaborated* to a System F-ish IR, defined very similar to the core calculus of *Tigris*, as:

```

1  inductive FExpr where
2  | CI      (i : Int)    (ty : MLType)
3  | CS      (s : String) (ty : MLType)
4  | CB      (b : Bool)   (ty : MLType)
5  | CUnit   (ty : MLType)
6  | Var     (x : String) (ty : MLType)           -- monomorphic view at site
7  | Fun     (param : String) (paramTy : MLType)
8           (body : FExpr) (ty : MLType)         -- paramTy → body.ty = ty
9  | App     (f : FExpr) (a : FExpr) (ty : MLType) -- result type after application
10 | TyLam   (a : TV) (body : FExpr)              -- type abstraction
11 | TyApp   (f : FExpr) (arg : MLType)            -- type application
12 | Let     (binds : Array (String × Scheme × FExpr)) -- each binding elaborated & wrapped
13           (body : FExpr) (ty : MLType)
14 | Prod'   (l r : FExpr) (ty : MLType)
15 | Cond    (c t e : FExpr) (ty : MLType)
16 | Match   (discr : Array FExpr)
17           (branches : Array (Array Pattern × FExpr))
18           (resTy : MLType)
19           (counterexample : Option (List Pattern))
20           (redundantRows : Array Nat)
21 | Fix     (e : FExpr) (ty : MLType)             -- rec tag
22 | Proj    (src : FExpr) (mname : String)
23           (idx : Nat) (ty : MLType)             -- field projection
24
deriving Repr, Inhabited

```

Using a pretty-printing algorithm similar to Wadler’s, FExpr is pretty printed to have a syntax that is very much similar (except it always denotes signatures and type application) to a subset of *Tigris*. This algorithm is used for subsequent IRs.

From there, we begin the lowering process by lowering it to *Lambda* IR.

Type system, together with typechecking, is a major part of *Tigris*’ implementation, we’ll have a more thorough discussion below.

3 Type System of the *Tigris* language

Tigris implements a constraint-based Hindley-Milner type system extended with

Predicates are of the form $C\bar{\alpha}$ where C is a class head. These are collected during inference and resolved via dictionary passing after constraint solving.

Strict Higher-Kinded Types refers to the fact that higher-kinded TVs must be fully applied. This is done through the use of KApp which is just enough for abstractions like Functor and Monad, see [footnote 7](#).

Limited Rank-N polymorphism refers to the fact that *schemes* (TSch) can appear inside monotypes $MType$, but the system remains in rank-1 polymorphism through the addition of skolemization during type ascription checking.²⁸ This means when checking $e : \sigma$, we instantiate σ with skolem constants²⁹ and verify the inferred type unifies. Intuitively and informally this should be sound as

- skolem-TVs are rigid in the sense that they are unique and cannot be unified with other types.
- skolem-TVs are tracked through a rigid set that is used to reject substitutions which would otherwise escape scope.

Constraint generation & Solving Generation of constraints are handled by inference, which in turn, solves them in one pass. The solver returns a substitution (also called a *solution*) and residual predicates that become part of the generalized scheme.

Let-Polymorphism refers to let-bound variables are generalized over TVs such that $tv \notin \text{ftv } \Gamma$, with predicate constraints becoming part of the scheme's context. Informally, it is correct to state that, program is still typeable even if users may not provide any signature due to the principality of HM systems.

3.0.1 Formal Grammar

The type system is built on its core calculus, an augmented version of the lambda calculus. To describe its typing discipline, we first present the formal grammar of the core calculus as well as the type expression, among other things.³⁰

$expr, e$	$::=$	term
	$const$	constant
	$tname$	variable
	$\lambda x. e$	lambda abstraction
	$\text{let } x_1 = e_1 \text{ and } \dots \text{ and } x_n = e_n \text{ in } e'$	letexp
	$e e'$	function application
	(e_1, e_2)	product
	$\text{if } e_1 \text{ then } e_2 \text{ else } e_3$	conditional
	$\text{fix } e$	fixpoint
	$\text{match } e_1, \dots, e_n \text{ with } Pmatrix$	pattern matching
	$e : sch$	type ascription

$pattern, pat$	$::=$	patterns
	$tname$	variable
	$_$	wildcard
	$const$	constant
	(pat_1, pat_2)	product
	$ctor pat_1 \dots pat_n$	constructor application

$Pmatrix$	$::=$	pattern matrix
	$ pb_1 \dots pb_m $	

$patbranch, pb$	$::=$	pattern branch
	$pat_1, \dots, pat_n \rightarrow expr$	

²⁸as **gen** or **generalize** does not generate higher-rank binders.

²⁹encoded by TCon i.e. a nullary constructor

³⁰most grammar, rules, are generated from report.ott using Ott.

$$\begin{array}{lcl}
 \text{tyctor} & ::= & \\
 & | & \text{upperident} \\
 & | & \text{Int} \\
 & | & \text{Bool} \\
 & | & \text{String} \\
 & | & \text{Unit}
 \end{array}$$

$$\begin{array}{lcl}
 \text{tyvar}, \text{tv}, \text{a}, \text{b} & ::= & \\
 & | & \text{lowerident}
 \end{array}$$

$$\begin{array}{lcl}
 \text{tvset}, \bar{\alpha} & ::= & \text{type variable set} \\
 & | & \emptyset \\
 & | & \{\text{tv}\} \\
 & | & \bar{\alpha}_1 \cup \bar{\alpha}_2 \\
 & | & \bar{\alpha}_1 \cap \bar{\alpha}_2 \\
 & | & \bar{\alpha}_1 \setminus \bar{\alpha}_2 \\
 & | & \text{ftv}(\Gamma) \\
 & | & \text{ftv}(\text{ty}) \\
 & | & \text{ftv}(\bar{\pi}) \\
 & | & \text{ftv}(\text{pred})
 \end{array}$$

$$\begin{array}{lcl}
 \text{tyargs}, \text{tys} & ::= & \text{one or more type arguments} \\
 & | & \text{ty} \\
 & | & \text{ty tys}
 \end{array}$$

$$\begin{array}{lcl}
 \text{typeexpr}, \text{ty}, \tau & ::= & \text{type expression} \\
 & | & \text{tyctor} \quad \text{nullary type constructor} \\
 & | & \text{tv} \quad \text{type variable} \\
 & | & \text{ty} \rightarrow \text{ty}' \quad \text{arrow type} \\
 & | & \text{ty} \times \text{ty}' \quad \text{product type} \\
 & | & \text{tyctor tys} \quad \text{nominal type application} \\
 & | & \kappa \text{ tys} \quad \text{higher-kinded TV head application}
 \end{array}$$

$$\begin{array}{lcl}
 \text{predicate}, \text{pred}, \pi & ::= & \text{predicate} \\
 & | & \text{ctor ty}_1 \dots \text{ty}_n
 \end{array}$$

$$\begin{array}{lcl}
 \text{predicates}, \text{preds}, \bar{\pi} & ::= & \\
 & | & \emptyset \\
 & | & \text{pred} \\
 & | & \text{pred}, \bar{\pi} \\
 & | & \bar{\pi}_1, \bar{\pi}_2
 \end{array}$$

$$\begin{array}{lcl}
 \text{scheme}, \text{sch}, \sigma & ::= & \text{scheme} \\
 & | & \forall \text{tv}_1 \dots \text{tv}_n. \bar{\pi} \Rightarrow \text{ty} \\
 & | & (\bar{\alpha}. \bar{\pi} \Rightarrow \text{ty}) \quad \text{from tvset} \\
 & | & \text{ty} \quad \text{Monotype}
 \end{array}$$

$$\begin{array}{lcl}
 \text{context}, \Gamma & ::= & \text{typing environment} \\
 & | & \emptyset \\
 & | & \Gamma, x : \text{sch} \\
 & | & x : \text{sch}
 \end{array}$$

$constraint, c$	$::=$	constraint
	$ty_1 \sim ty_2$	equality constraint
	$pred_{evidx}$	predicate constraint with evidence
$constraints, C$	$::=$	constraint set
	$\emptyset$	
	c	
	C_1, C_2	set join
$subst, \theta$	$::=$	substitution
	$\emptyset$	
	$tv \mapsto ty$	
	$\theta_1 \circ \theta_2$	composition
$evidence, ev$	$::=$	evidence term
	x	evidence variable
	$ctor\ ev_1 .. ev_n$	dictionary constructor
$evctx, \Delta$	$::=$	evidence context
	$\emptyset$	
	$\Delta, evidx : pred$	

3.0.2 Judgments & Inference rules

We now, present the judgments as well as the inference rule. Each judgement comes with a remark that informally explains the implementation.

Unification computes a most general unifier (MGU) for equality constraints. The unification engine

- checks occurrence to prevent infinite types as shown in [section 2.4](#)
- requires constructors to match exactly
- unifies KApp with concrete type applications TApp, provided arities match.
- α -renames schemes before unifying them structurally.

$$\boxed{\theta = \text{unify}(ty_1, ty_2)}$$

(Unification of types)

U-REFL	U-VARL	U-VARR
$\frac{}{\emptyset = \text{unify}(ty, ty)}$	$\frac{tv \notin \text{ftv}(ty)}{tv \mapsto ty = \text{unify}(tv, ty)}$	$\frac{tv \notin \text{ftv}(ty)}{tv \mapsto ty = \text{unify}(ty, tv)}$
U-ARROW	U-PROD	
$\frac{\theta_1 = \text{unify}(a, a') \quad \theta_2 = \text{unify}([\theta_1]b, [\theta_1]b')}{\theta_2 \circ \theta_1 = \text{unify}(a \rightarrow b, a' \rightarrow b')}$	$\frac{\theta_1 = \text{unify}(a, a') \quad \theta_2 = \text{unify}([\theta_1]b, [\theta_1]b')}{\theta_2 \circ \theta_1 = \text{unify}(a \times b, a' \times b')}$	

Constraint Solving are done left-to-right, threading the substitution through. Predicate constraints are collected and returned as residuals for instance resolution.

$$\boxed{(\theta, \bar{\pi}) = \text{solve}(C)} \quad (\text{Constraint solving})$$

$$\begin{array}{c} \text{SLV-EMPTY} \\ \hline (\emptyset, \emptyset) = \text{solve}(\emptyset) \end{array} \quad \begin{array}{c} \text{SLV-EQ} \\ \theta = \text{unify}(ty_1, ty_2) \\ \hline (\theta', \bar{\pi}) = \text{solve}([\theta]C) \\ \hline (\theta' \circ \theta, \bar{\pi}) = \text{solve}(ty_1 \sim ty_2, C) \end{array} \quad \begin{array}{c} \text{SLV-PRED} \\ \hline (\theta, \bar{\pi}) = \text{solve}(C) \\ \hline (\theta, [\theta]pred, \bar{\pi}) = \text{solve}(pred_{\text{evidx}}, C) \end{array}$$

Instantiation replaces bound TVs with fresh TVs and returns the predicate constraints that must be satisfied.

$$\boxed{(ty, \bar{\pi}) = \text{inst}(sch)} \quad (\text{Scheme instantiation})$$

$$\begin{array}{c} \text{INST-FORALL} \\ tv'_1 = \text{fresh} \quad \dots \quad tv'_n = \text{fresh} \\ \theta = \{tv_1 \mapsto tv'_1, \dots, tv_n \mapsto tv'_n\} \\ \hline ([\theta]ty, [\theta]\bar{\pi}) = \text{inst}(\forall tv_1 \dots tv_n. \bar{\pi} \Rightarrow ty) \end{array}$$

Generalization abstract over TVs not free in the environment. Predicates mentioning such TVs become part of the context, that is,

A predicate is kept if and only if it mentions one or more qTVs³¹ that also appears in the result type.

$$\boxed{sch = \text{gen}(\Gamma, ty, \bar{\pi})} \quad (\text{Generalization})$$

$$\begin{array}{c} \text{GEN-GEN} \\ \bar{\alpha} = (\text{ftv}(ty) \cup \text{ftv}(\bar{\pi})) \setminus \text{ftv}(\Gamma) \\ \hline (\bar{\alpha}. \bar{\pi} \Rightarrow ty) = \text{gen}(\Gamma, ty, \bar{\pi}) \end{array}$$

Expression Typing generates constraints that are used in constraint solving to determine³² the actual type. The judgment $\Gamma \vdash e \Rightarrow ty; C$ reads:

Under environment Γ , expression e has type ty generating constraints C .

Based on the core calculus, inference applies to:

Variables by looking up in Γ and instantiate the scheme,

Constants which already have concrete types,

Lambda abstraction of which parameter³³ gets fresh TVs,

Application by generating constraints that make the function type match,

Product by generating constraints that make the product type match,

³²by applying all substitutions

³³a lambda abstraction only takes one (1) parameter, as mentioned before.

Conditional by requiring then/else-branch to agree, condition to be Bool.

Let by inferring body *after* inferring/solving/generalizing RHS,

Let rec by extending Γ with fresh TVs for all binders *before* inferring each RHS, whose generated constraints equate these with inferred types,

Fix where **fix** $\lambda f. e$ has type of e ,

Ascription by checking that inferred type is at least as general, which, takes different approach based on ranks where

- for rank-1 ascriptions, we simply unify.
- for rank- n (where $n \geq 2$, that is, if $TSch$ contains nested quantifiers), we have the following steps:
 1. **Instantiation** instantiate the scheme $\sigma = \forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau$ with fresh type variables $\bar{\beta}$, yielding τ_{inst} .
 2. **Constraint solving** solve the current constraints augmented with $\tau_e \sim \tau_{\text{inst}}$ to obtain a substitution θ' .³⁴
 3. **Rigidity check** For each binding $\alpha \mapsto \tau'$ in the substitutions θ' , if α is in the current rigid set $\mathcal{R}$, then τ' must equal α (from the property of rigids. i.e. it must *not* be specialized). Failing this check results in the ascription being rejected with a unification error.
 4. **Skolemization** As stated above and below, ensures the generality does match the declared (polymorphic) type.

Pattern matching where for

- each branch i : Patterns $p_{i,1}, \dots, p_{i,m}$ are checked against the corresponding discriminant types. Γ is then enriched with pattern bindings. TVs arising from patterns are marked as *rigid* (within their branches) which prevents leaking³⁵ across branches. RHS is inferred after, requiring all branches to agree (through constraints).
- discriminants: Each one e_j is inferred independently, yielding types $\tau_1, \dots, \tau_m$. whose types must agree with pattern types.

Additionally, pattern exhaustiveness and redundancy are checked separately (via `Exhaustive.exhaustWitness`) but not reflected in the typing judgment. Non-exhaustive matches or redundant branches generate warnings but are not type errors. Note that the latter is immediately removed from the matrix.

$$\boxed{\Gamma \vdash e \Rightarrow ty; C} \quad (Constraint\text{-}based\ inference)$$

I-VAR

$$\frac{x : sch \in \Gamma \quad (ty, \bar{\pi}) = \text{inst}(sch)}{\Gamma \vdash x \Rightarrow ty; C}$$

I-INT

$$\Gamma \vdash \text{intlit} \Rightarrow \text{Int}; \emptyset$$

I-STRING

$$\Gamma \vdash \text{strlit} \Rightarrow \text{String}; \emptyset$$

I-TRUE

$$\Gamma \vdash \text{true} \Rightarrow \text{Bool}; \emptyset$$

³⁴This can be seen as a premature solving. We *tentatively* solve them only to use them in the next step that is rigidity check.

³⁵this is mostly a precaution. The “leaking” described here is only likely to happen when pattern matching index families, which, is not supported, at all.

$\text{I-FALSE} \frac{}{\Gamma \vdash \mathbf{false} \Rightarrow \mathbf{Bool}; \emptyset}$	$\text{I-UNIT} \frac{}{\Gamma \vdash () \Rightarrow \mathbf{Unit}; \emptyset}$	$\text{I-LAM} \frac{tv = \mathbf{fresh} \quad \Gamma, x : tv \vdash e \Rightarrow ty; C}{\Gamma \vdash \lambda x. e \Rightarrow tv \rightarrow ty; C}$
$\text{I-APP} \frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2 \quad tv = \mathbf{fresh}}{\Gamma \vdash e_1 e_2 \Rightarrow a; (C_1, C_2), ty_1 \sim ty_2 \rightarrow a}$	$\text{I-PROD} \frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2}{\Gamma \vdash (e_1, e_2) \Rightarrow ty_1 \times ty_2; C_1, C_2}$	
$\text{I-COND} \frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2 \quad \Gamma \vdash e_3 \Rightarrow ty_3; C_3}{\Gamma \vdash \mathbf{if} e_1 \mathbf{then} e_2 \mathbf{else} e_3 \Rightarrow ty_2; (((C_1, C_2), C_3), ty_1 \sim \mathbf{Bool}), ty_2 \sim ty_3}$		
$\text{I-LET} \frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad (\theta, \bar{\pi}) = \mathbf{solve}(C_1) \quad sch = \mathbf{gen}(\Gamma, [\theta]ty_1, \bar{\pi}) \quad \Gamma, x : sch \vdash e_2 \Rightarrow ty_2; C_2}{\Gamma \vdash \mathbf{let} x = e_1 \mathbf{in} e_2 \Rightarrow ty_2; [\theta]C_2}$	$\text{I-FIX} \frac{tv = \mathbf{fresh} \quad \Gamma, f : tv \vdash e \Rightarrow ty; C}{\Gamma \vdash \mathbf{fix}(\lambda f. e) \Rightarrow ty; C, tv \sim ty}$	$\text{I-LETREC} \frac{tv = \mathbf{fresh} \quad \Gamma, x : (tv) \vdash e_1 \Rightarrow ty_1; C_1 \quad (\theta, \bar{\pi}) = \mathbf{solve}(C_1) \quad sch = \mathbf{gen}(\Gamma, [\theta]ty_1, \bar{\pi}) \quad \Gamma, x : sch \vdash e \Rightarrow ty; C}{\Gamma \vdash \mathbf{let} x = e_1 \mathbf{in} e \Rightarrow ty_1; [\theta]C}$

Since match clause and type ascription are not easy to correctly type in OTT and too convoluted in terms of readability, we manually type them as follows:³⁶

$\Gamma \vdash e_j \Rightarrow \tau_j; C_j \quad (\forall j \in 1..m)$	
$\alpha = \mathbf{fresh}$	
$\text{For each branch } i \in 1..k : \quad (\bar{p}_i) = m \quad \Gamma \vdash p_{i,j} \Leftarrow \tau_j \Rightarrow \Gamma_{i,j}; C_{p_{i,j}} \quad (\forall j \in 1..m)$	
$\Gamma_i = \Gamma_{i,m} \quad (\text{final extended environment for branch } i)$	
$\bar{\beta}_i = \mathbf{ftv}(\text{bound variables in } \bar{p}_i) \quad \mathbf{pushRigid}(\bar{\beta}_i) \quad \Gamma_i \vdash r_i \Rightarrow \tau_{r_i}; C_{r_i} \quad \mathbf{popRigid}()$	
$C_{\mathbf{unify}} = \{\tau_{r_i} \sim \alpha \mid i \in 1..k\} \quad C = \bigcup_{j=1}^m C_j \cup \bigcup_{i=1}^k \left(\bigcup_{j=1}^m C_{p_{i,j}} \cup C_{r_i} \right) \cup C_{\mathbf{unify}}$	
$\Gamma \vdash \mathbf{match} e_1, \dots, e_m \mathbf{with} \mid p_{1,1}, \dots, p_{1,m} \rightarrow r_1 \Rightarrow \alpha; C$	I-MATCH
$\frac{\Gamma \vdash e \Rightarrow \tau_e; C_e}{\Gamma \vdash (e : \tau) \Rightarrow \tau; C_e, \tau_e \sim \tau} \text{I-ASCRIBE-MONO}$	
$\frac{\Gamma \vdash e \Rightarrow \tau_e; C_e \quad \sigma = \forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau \quad \bar{\beta} = \mathbf{fresh}^{ \bar{\alpha} } \quad \theta = [\bar{\alpha} \mapsto \bar{\beta}] \quad \tau_{\mathbf{inst}} = \theta(\tau) \quad C_0 = \text{current constraints} \quad (\theta', _) = \mathbf{solve}(C_0, \tau_e \sim \tau_{\mathbf{inst}}) \quad \mathcal{R} = \text{current rigid set} \quad \forall (\alpha \mapsto \tau') \in \theta'. \alpha \in \mathcal{R} \implies \tau' = \alpha \quad \tau_{\mathbf{sk}} = \mathbf{skolem}(\sigma) \quad \text{add constraints for } \bar{\pi}}{\Gamma \vdash (e : \sigma) \Rightarrow \tau_{\mathbf{inst}}; C_e, \tau_e \sim \tau_{\mathbf{inst}}} \text{I-ASCRIBE-POLY}$	

³⁶Readers shall note that because these rules are hand-typed, some symbols are not defined as a production rule above, or a formula in OTT, in this case, adequate natural language is used to define/describe them inline. The author has no one but himself to blame due to his unfamiliarity with OTT.

Patterns bind variables and generate constraints from the discriminants. The judgment $\Gamma \vdash pat \Rightarrow \Gamma'; C; ty$ reads:

Pattern pat against type ty extends Γ to Γ' , generating constraints C .

$\Gamma \vdash pat \Rightarrow \Gamma'; C; ty$			(Pattern inference)
P-WILD $\frac{}{\Gamma \vdash _ \Rightarrow \Gamma; \emptyset; ty}$	P-VAR $\frac{x \notin \text{dom}(\Gamma)}{\Gamma \vdash x \Rightarrow \Gamma, x : ty; \emptyset; ty}$	P-CONSTINT $\frac{}{\Gamma \vdash \text{intlit} \Rightarrow \Gamma; ty \sim \text{Int}; ty}$	
P-CONSTSTR $\frac{}{\Gamma \vdash \text{strlit} \Rightarrow \Gamma; ty \sim \text{String}; ty}$	P-CONSTTRUE $\frac{}{\Gamma \vdash \text{true} \Rightarrow \Gamma; ty \sim \text{Bool}; ty}$	P-CONSTFALSE $\frac{}{\Gamma \vdash \text{false} \Rightarrow \Gamma; ty \sim \text{Bool}; ty}$	
P-CONSTUNIT $\frac{}{\Gamma \vdash () \Rightarrow \Gamma; ty \sim \text{Unit}; ty}$		P-PROD $\frac{\begin{array}{l} a = \text{fresh} \quad b = \text{fresh} \\ \Gamma \vdash pat_1 \Rightarrow \Gamma_1; C_1; a \\ \Gamma_1 \vdash pat_2 \Rightarrow \Gamma_2; C_2; b \end{array}}{\Gamma \vdash (pat_1, pat_2) \Rightarrow \Gamma_2; (C_1, C_2), ty \sim a \times b; ty}$	

Since constructor applications are not easy to correctly type in OTT and too convoluted in terms of readability, we manually type them as follows:

$$\frac{\begin{array}{l} ctor : \sigma \in \Gamma \quad (\tau_{ctor}, \bar{\pi}) = \text{inst}(\sigma) \quad \text{peel}(\tau_{ctor}) = (\tau_1, \dots, \tau_n, \tau_{\text{res}}) \quad n = |\bar{p}| \\ \Gamma_0 = \Gamma \quad \Gamma_{i-1} \vdash p_i \Leftarrow \tau_i \Rightarrow \Gamma_i; C_i \quad (\forall i \in 1..n) \quad C_\pi = \bar{\pi} \end{array}}{\Gamma \vdash ctor \ p_1 \dots p_n \Leftarrow \tau \Rightarrow \Gamma_n; C_1, \dots, C_n, C_\pi, \tau \sim \tau_{\text{res}}} \text{ P-CTOR}$$

Where peel decomposes an arrow type into its argument types and result type:

$$\text{peel}(\tau_1 \rightarrow \tau_2 \rightarrow \dots \rightarrow \tau_n \rightarrow \tau_r) = (\tau_1, \tau_2, \dots, \tau_n, \tau_r)$$

The constructor scheme σ is looked up in Γ and instantiated with fresh type variables. Predicate constraints from instantiation are added to ensure typeclass requirements are propagated.

Below, we list some remarks for concepts mentioned above that are not properly introduced.

Remark (Skolemization). replaces TVs with skolems that cannot unify with anything except themselves.

$$\text{skolem}(\forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau) = [\bar{\alpha} \mapsto \overline{\text{sk}_{\bar{\alpha}}}] (\tau)$$

where each sk_{α} is a fresh skolem constant (implemented as a special type constructor, e.g., $\text{?sk}.\alpha$ in the implementation).

The word *rigid* is to capture the key nature of skolems: $\text{sk}_{\alpha} \sim \tau$ succeeds iff $\tau = \text{sk}_{\alpha}$.

Remark (Rigid Set/Stack). The type system maintains a stack of rigid type variable sets to handle nested scopes correctly.

$$\begin{array}{ll} \text{pushRigid}(\bar{\alpha}) : & \mathcal{R} := \mathcal{R} \cup \bar{\alpha}; \quad \text{stack} := \bar{\alpha} :: \text{stack} \\ \text{popRigid}() : & \text{let } (S :: \text{rest}) = \text{stack} \text{ in } \mathcal{R} := \mathcal{R} \setminus S; \quad \text{stack} := \text{rest} \end{array}$$

Invariant. At any point during inference, the rigid set maintains the fact that

$$\mathcal{R} \equiv \bigcup \text{stack}$$

(i.e. it is the union of all sets currently on the stack.) When solving constraints, any substitution $\alpha \mapsto \tau$ where $\alpha \in \mathcal{R}$ and $\tau \neq \alpha$ is rejected.

Note that the $(:=)$ operation is implemented by the ST monad, through the `MonadStateOf` interface.

Remark (Solving Rigids). Though not shown above in the inference rule, the constraint solver respect rigid TVs:

$$\frac{\alpha \in \mathcal{R} \quad \tau \neq \alpha}{\text{solve}(\alpha \sim \tau) = \mathbf{error}} \text{ SOLVE-RIGID} \qquad \frac{\alpha \in \mathcal{R}}{\text{solve}(\alpha \sim \alpha) = \emptyset} \text{ SOLVE-RIGID-OK}$$

Remark (Soundness). The type system should maintain soundness through several mechanisms:

Skolems During pattern matching and typing ascription, some TVs are marked as *rigid* and tracked in τTV . Substitutions that would specialize them to incompatible types are rejected by the solver. This is thought to prevent unsound generalizations.

Rank-N skolemization When checking $e : \forall tv.\pi \Rightarrow ty$, we replace tv with a fresh skolem constant (in order to make it truly general) to ensure expressions do work for all instantiations.

Occurs check is used to prevent users from constructing equirecursive types such as $\alpha \sim \alpha \rightarrow \beta$ or $\alpha \sim \alpha \times \beta$ arising from codes such as `fun x => x x` or `fun | (x, y) => x | z => z`, respectively.

Hygienic generalization Only TVs not free in Γ are generalized. Predicates are kept based on the condition described above that shall avoid vacuous constraints.

Syntax restrictions Lastly, if one looks closely at the BNF grammar of type expression or the formal grammar presented above, they will find that, it is actually not possible to write $(\forall a.a \rightarrow a) -> \text{Int} \times \text{Bool}$ as though `MType` embeds scheme, the parsing facility does not admit schemes inside monotypes. Indeed, the only way to introduce embedded scheme is through polymorphic fields in datatype declarations, which, is not harmful. This is perhaps, the strongest restriction that is thought to guarantee the soundness of the type system.

Remark (Counterexamples: The need of rigids). Below we provide some counterexamples that emphasize the need of rigids:

- **(Wrong Specialization)** Consider the following

```
let id = (fun x => x : forall a, a -> a) 1

-- [ERROR] Can't unify type Int with ?sk.a.
let id'   : forall a, a -> a = fun x => (x : Int)
let id'' x : forall a, a -> a = x + 1
```

The first example is acceptable: The ascription claims polymorphism, but the body is monomorphic, which is fine since we are instantiating the polymorphic function.

In the second example however, It would be unsound for a , introduced as a rigid inside `fun x => ...` to be specialized to `Int`. Same applies to the third.

- **(Polymorphism recovery)** Typically, typeclass methods (which is very likely to be polymorphic themselves) are used directly which allows the special treatment of them like in Haskell '98. However, one may wish to write:

```

type Functor (f : 1) = Functor (forall a b, (a -> b) -> f a -> f b)

let prog (Functor f) =
  let l = 1 :: 2 :: 3 :: Nil in
  and l' = "1" :: "2" :: "3" :: Nil in
  and const k _ = k in
  let x = f (_ + 1) l in
  and y = f (const ()) l'
  in (x, y)

```

This is perfectly acceptable, yet, do note the importance of rigids: a and b are rigid within each use, within each call. This allows the independent instantiation of them as f is specialized to different concrete type at x ($a \mapsto \text{Int}, b \mapsto \text{Int}$) and y ($a \mapsto \text{String}, b \mapsto \text{Unit}$).

3.0.3 Instance resolution

Merely associating instances with typeclasses is not enough at all: We, or, to be precise, the compiler, requires a general procedure (algorithm) to lookup the right instance for the right type of some class.³⁷ This process is called *instance resolution*. The algorithm described below searches the instance environment for a matching instance, recursively resolving any context constraints.

Instance metadata Each registered instance I 's metadata is a structure

$$I = \langle \text{iname}, \text{cls}, \bar{\alpha}, \bar{\pi}_{\text{ctx}} \rangle$$

where

- iname is the instance's unique identifier (naming convention: i_Eq_0),
- cls is the class head,
- $\bar{\alpha}$ is the list of type arguments in the instance head,
- $\bar{\pi}_{\text{ctx}}$ is the list of context predicates.

e.g. an instance declaration `instance : forall  $\alpha$  [Eq  $\alpha$ ], Eq (List  $\alpha$ ) = ...` produces

$$I = \langle \text{i_Eq_1}, \text{Eq}, [\text{List } \alpha], [\text{Eq } \alpha] \rangle$$

Head Unification refers to the procedure of checking whether a goal $C \bar{\tau}$ can match an instance head $C \bar{\alpha}$ before attempting full resolution.

³⁷Or as in PL jargon: finding a *dictionary* (evidence term) that *witnesses* a predicate constraint $\pi = C \bar{\tau}$.

Algorithm 1 Head Unification

```
1: function UNIFYHEAD( $\bar{\tau}_{\text{goal}}, \bar{\alpha}_{\text{inst}}$ )
2:   if  $|\bar{\tau}_{\text{goal}}| \neq |\bar{\alpha}_{\text{inst}}|$  then
3:     return error
4:   end if
5:    $\theta \leftarrow \emptyset$ 
6:   for  $i \leftarrow 1$  to  $|\bar{\tau}_{\text{goal}}|$  do
7:      $\theta' \leftarrow \text{unify}(\text{apply}(\theta, \tau_i), \text{apply}(\theta, \alpha_i))$ 
8:      $\theta \leftarrow \theta' \circ \theta$ 
9:   end for
10:  return  $\theta$ 
11: end function
```

Resolution Algorithm The main resolution algorithm maintains a *visited set* $\mathcal{V}$ to detect cycles (which indicates either ambiguous or divergent instance resolution).

Algorithm 2 Instance Resolution

```

1: function RESOLVE( $\Gamma, \pi, \mathcal{V}$ )
2:   Input: Environment  $\Gamma$ , predicate  $\pi = C \bar{\tau}$ , visited set  $\mathcal{V}$ 
3:   Output: Evidence expression  $e$  s.t.  $e : C \bar{\tau}$ , or error
4:
5:   if  $\pi \in \mathcal{V}$  then
6:     return error: “cyclic instance resolution”
7:   end if
8:    $\mathcal{V} \leftarrow \mathcal{V} \cup \{\pi\}$ 
9:
10:   $insts \leftarrow \Gamma.instInfo[C]$  ▷ All instances for class  $C$ 
11:  if  $insts = \emptyset$  then
12:    return error: “no instance for  $\pi$ ”
13:  end if
14:
15:   $found \leftarrow \text{none}$ 
16:  for each  $I = \langle iname, C, \bar{\alpha}, \bar{\pi}_{ctx} \rangle$  in  $insts$  do
17:    try:
18:       $\theta \leftarrow \text{UNIFYHEAD}(\bar{\tau}, \bar{\alpha})$ 
19:       $\bar{\pi}'_{ctx} \leftarrow \text{apply}(\theta, \bar{\pi}_{ctx})$ 
20:      for each  $\pi' \in \bar{\pi}'_{ctx}$  do
21:         $\_ \leftarrow \text{RESOLVE}(\Gamma, \pi', \mathcal{V})$  ▷ Ensure solvable
22:      end for
23:       $e \leftarrow (iname : C \bar{\tau})$  ▷ Ascribed instance reference
24:      if  $found \neq \text{none}$  then
25:        return error: “ambiguous instance for  $\pi$ ”
26:      end if
27:       $found \leftarrow \text{some}(e)$ 
28:    catch: continue ▷ Try next instance
29:  end for
30:
31:  if  $found = \text{none}$  then
32:    return error: “no matching instance for  $\pi$ ”
33:  end if
34:  return  $found$ 
35: end function

```

Remark (ResolveM). The algorithm described above runs inside **ResolveM** monad. Just like any monad we previously mentioned, it is a ST backed state monad with error handling (hence EST), whose state is `ResolveState`, which is precisely the visited set $\mathcal{V}$.

```

abbrev ResolveState := Std.HashSet Pred
abbrev ResolveM  $\sigma$  := StateRefT ResolveState (EST TypingError  $\sigma$ )

variable { $\sigma$  : Type}

instance : MonadLift (Except TypingError) (EST TypingError  $\sigma$ ) where
  monadLift
  | .error e  $\Rightarrow$  throw e
  | .ok e  $\Rightarrow$  return e

```

Meanwhile, the `MonadLift` instance defines a natural transformation from `Except` to `EST`, lifting fail-able but pure actions to run in it.

Remark (Properties of instance resolution). The key design points of this algorithm are described below:

Termination has not been proved, as noted by the partial modifier in the source. Intuitively, It terminates if the dependency graph of a instance is well-founded i.e. does not contain infinite chains of instance dependencies. The set $\mathcal{V}$ (line 5) detects direct cycles, but may not (or does not) guarantee termination for all pathological instances.

Semantic coherence is enforced through a *single-match* policy: If multiple instances could match a goal (line 24), resolution fails with an ambiguity error, which, guarantees that the meaning of a program does not depend on which instance is arbitrarily chosen. However, users should note that, contrary to the usual practice, this restriction applies to all instances regardless of their generality, that is, the more specific one would not be chosen although it may make more sense doing so, and will result in a compile-time error. To allow the existence of multiple matched instances, one way is to introduce a precedence and a default rule like the above practice, but this currently is not supported.

Evidence generation refers to ascribed instance identifiers:

$$e = (\text{iname} : C \ \bar{\tau})$$

Which is later *elaborated* into dictionary-passing style during the System F IR translation phase from which is then encoded in the field projection node `Proj src mname i ty`.

Context propagation refers to instantiating context predicates with the unifying substitution (line 19) and the recursively resolving of them (line 20), from instances with context predicates $\bar{\pi}_{\text{ctx}}$ (e.g., `Eq a in instance : forall  $\alpha$  [Eq  $\alpha$ ], Eq (List  $\alpha$ ) = ...`)

We shall simulate this algorithm with a simple example below: Consider resolving `Eq (List Int)` with instances:

$$\begin{aligned} I_0 &= \langle \text{i_Eq_0}, \text{Eq}, [\text{Int}], \emptyset \rangle, \\ I_1 &= \langle \text{i_Eq_1}, \text{Eq}, [\text{List } \alpha], [\text{Eq } \alpha] \rangle. \end{aligned}$$

1. Goal: `Eq (List Int)`
2. Try I_0 : `unify(List Int, Int)` fails
3. Try I_1 : `unify(List Int, List  $\alpha$ )` succeeds with $\theta = [\alpha \mapsto \text{Int}]$
4. Context: `apply( $\theta$ , [Eq  $\alpha$ ]) = [Eq Int]`
5. Recursively resolve `Eq Int`:
 - (a) Try I_0 : `unify(Int, Int)` succeeds with $\theta = \emptyset$
 - (b) Context: $\emptyset$, no recursion needed
 - (c) Return `(i_Eq_0 : Eq Int)`
6. Return `(i_Eq_1 : Eq (List Int))`

Remark (Comparison with GHC). GHC implements a more robust, sophisticated typeclass facility. Although *Tigris* has learned and taken many inspirations from it, its instance resolution strategy is much simpler than GHC's:

- Tigris does not allow overlapping instances, regardless of any ordering, whereas GHC provides precedence that controls the granularity of instance resolution;
- Tigris does not have instance chains as there is no mechanism for specifying fallback instances or instance priority

However, when compared to Haskell '98 or 2010, *Tigris*' instance declarations is somewhat more expressive as we have some form of FlexibleContexts³⁸ as it is possible to define **instance** $\forall a$ [Eq a], Eq ($a * a$) = ..; Additionally, multi-parameter classes are also supported, such as the HAdd and HEq class mentioned above.

3.0.4 System F elaboration

The typed syntax TExpr, after instance resolution, is elaborated to FExpr. Below, we discuss an informal description of the elaboration.

Given a scheme

$$\forall \alpha_1 \cdots \alpha_n [P_1, \dots, P_k], \tau$$

we elaborate a binding rhs to

$$\Lambda \alpha_1, \dots, \alpha_n. \lambda d_1, \dots, d_k. \text{body}$$

where $d_1, \dots, d_k$ are the respective instances of preds $P_1, \dots, P_k$. Except when the rhs already starts with the exact k dictionary lambdas, that is, wrappers auto-generated for class methods, in which case we only add TyLams.

At each Var occurrence:

$$\text{Var } x : \sigma \text{ where } \sigma = \forall \bar{\alpha} \bar{P}, \text{tbody}$$

is elaborated to

$$\begin{aligned} & \text{TyApp } \cdots (\text{TyApp } (\text{Var } x_{\text{ty}_{\text{mono}}}) \alpha_1) \cdots \alpha_n \\ & \text{App}(\cdots) d_1 \\ & \vdots \\ & \text{App}(\cdots) d_k \end{aligned}$$

A dictionary argument³⁹ is either

- in scope, that is, matching one of the predicate templates introduced by lambdas, or,
- synthesized by instance resolution, producing an untyped expression. Inference and elaboration is then re-run on that expression to transform it to an FExpr. This procedure is recursive.

In case if a method field type is polymorphic (e.g. Functor), as in TSch (Forall as [] τ), it is reified as a System F type abstractions by wrapping the projected field with TyLam, for each binder in as. This is thought to be canonical for Higher-ranked types. Do note, however, per-method predicate contexts inside **TSch** are currently unsupported and will be rejected.

³⁸An extension of GHC that lifts the Haskell 98 restriction that the type-class constraints (anywhere they appear) must have the form class type-variable or class (type-variable type1 type2 .. typen).

³⁹named $d_P.\text{cls}_i$ where P is the predicate, i is the instance counter for that class; a case it is memoized, the identifier is prepend with a r .

Dictionary memoization refers to the memoization of synthesized dictionaries. It is common for a single instance to be used multiple times, in which case it becomes rather inefficient, both compile-time and runtime as each encounter results in a newly synthesized dictionary, or a dictionary application of multiple dependent dictionaries (as is the case for instances with contexts). Memoization tracks the first usage of a dictionary at function-level, upon encountering multiple usage, the synthesized instance is reused, and the function body is prepended by a let-exp that binds this instance that *hoist* the inlined dictionary application. This is similar to common subexpression elimination, except in this case it is targeted and not general at all, which comes with the benefit of being more convenient to implement. Consider the hierarchy of Functor, Applicative and Monad, in a rather elaborated example:

```
class Applicative (f : 1) =
  { pure : ∀ a, a -> f a
    -- lazy to preserve eval semantics
    , seq : ∀ a b, f (a -> b) -> (Unit -> f a) -> f b }
class Monad (m : 1) =
  { bind : ∀ a b, m a -> (a -> m b) -> m b
    , return : ∀ a, a -> m a }

infixl 55 ">=>" => bind
instance ∀(m : 1) [Monad m], Applicative m =
  { pure = return
    , seq f x = f >=> fun f => x () >=> fun x => return $ f x
    }
instance ∀(f : 1) [Applicative f], Functor f =
  { map f x = seq (pure f) fun _ => x }

instance Monad Option =
  { return = Some
    , bind
      | None, _ => None
      | Some x, f => f x }

let seqOp = seq (Some id)
```

With `i_Monad_0`, `i_Applicative_0` corresponding to the `Option` instance of `Monad` and the general `Applicative` instance, the System F IR result is

```
let seqOp : ∀ α, (Unit → Option α) → Option α =
  let rd_Applicative_1 : Applicative Option =
    let rd_Monad_0 : Monad Option = i_Monad_0
    in i_Applicative_0@Option rd_Monad_0
  in (λ α. rd_Applicative_1[1, seq]@Option (Some@(α → α) id@α))
```

As shown above, memoization have come into effect as the application of dictionary is cached and is bound by `rd_Applicative_1`, which, has a counter of 1, this indicates that dictionary application creates (synthesizes) new dictionaries. Therefore it is much necessary to have memoization as it mainly boosts compile performance.

In practice, any elaboration runs inside the F monad, which, together with recursion state⁴⁰, is defined to be

⁴⁰It is worth noting that elaboration does not only depend on the state shown below, additional trivial environments are not shown here, please refer to `fsyntax.lean` and `fexpr.lean`.

```

structure FState where
  log      : Logger := ""
  memo     : Std.HashMap Pred (String × Scheme × FExpr) := ∅
  gensym   : Nat := 0
  deriving Inhabited

```

```

abbrev F := EStateM TypingError FState

```

A canonical state monad equipped with error reporting is used here, contrary to before: This is because of the many usage of local states in recursive call during elaboration. Stack-allocated (function parameters) states are much easier to work with in this case, than a *real* threadlocal mutable state.

Remark (SysF IR pretty-printed syntax). Here we have shown a more elaborated example of pretty-printed System F IR source. It should be obvious for anyone that is familiar with OCaml or Haskell, especially the type application syntax $\langle \text{term} \rangle @ \langle \text{type-expr} \rangle$. But for reference, we present the relevant pretty-printer implementation below:

```

1  local instance : Std.ToFormat Pattern where
2    format := .text [] Pattern.toString
3  open Std Std.Format in
4  partial def FExpr.unexpand : FExpr → Format
5    | .CI i _ | .CB i _ | .CS i _ ⇒ repr i
6    | .CUnit _ ⇒ format ()
7    | .App f a _ ⇒ parenL? f ++ group (indentD (parenR? a))
8    | .Proj src mname i _ ⇒ parenL? src ++ sbracket (format i ++ ", " ++ format mname)
9    | .Cond c t e _ ⇒
10     "if" ◇ unexpand c ↔ "then" ++ indentD (unexpand t)
11     ↔ "else" ++ indentD (unexpand e)
12    | .Fix e _ ⇒ "rec" ◇ unexpand e
13    | .Var x _ ⇒ format x
14    | .Prod' p q _ ⇒ paren $ unexpand p ◇ "," ◇ unexpand q
15    | .Fun param pTy body _ ⇒
16     group $ "fun" ◇ param ◇ ":" ◇ pTy.toString ◇ "⇒" ++ group (indentD (unexpand body))
17    | .Match discr branches .. ⇒
18     let discr := discr.map unexpand
19     let br := branches.map fun (pats, e) ⇒
20       "|" ◇ joinSep' pats ", " ◇ "⇒" ++ group (indentD (unexpand e))
21     group $ "match" ◇ joinSep' discr ", " ◇ "with" ↔ joinSep' br line
22    | .Let binds body _ ⇒
23     let (binds, recflag) := binds.foldl (init := ([], false)) fun (acc, recflag) (id, sch, fe) ⇒
24       match fe with
25       | .Fix (.Fun _ _ body _) _ ⇒
26         (acc.push $ id ◇ ":" ◇ toString sch ◇ group ("=" ++ indentD (unexpand body)), true)
27       | _ ⇒
28         (acc.push $ id ◇ ":" ◇ toString sch ◇ group ("=" ++ indentD (unexpand fe)), recflag)
29     let recStr := if recflag then .text " rec " else .text " "
30     "let" ++ recStr ++ joinSep' binds (line ++ "and ") ↔ "in" ◇ unexpand body
31    | .TyLam a body _ ⇒
32     let (tvs, body) := Helper.peelTyLam [a.elimStr] body
33     group $ paren $ "λ" ◇ joinSep tvs " " ++ "." ++ indentD (unexpand body)
34    | .TyApp f ty ⇒ parenL? f ++ "@" ++ parenT ty
35  where

```

```

36 parenR?
37   | p@(.App ..) | p@(.Fun ..) | p@(.Cond ..) | p@(.Let ..) | p@(.Match ..) ⇒ paren (unexpand p)
38   | p ⇒ unexpand p
39 parenL?
40   | p@(.Fun ..) | p@(.Cond ..) | p@(.Let ..) | p@(.Match ..) ⇒ paren (unexpand p)
41   | p ⇒ unexpand p
42 parent
43   | t@(TVar _) | t@(TCon _) | t@(TApp _ []) | t@(KApp _ []) ⇒ text t.toStr
44   | t ⇒ paren t.toStr

```

Remark (On the usefulness of System F IR). Since the beginning of this project/thesis, the author himself has a very clear goal of compiling *Tigris* to Common Lisp (SBCL). Since Lisp is a dynamically typed language with some form of type inference and propagation (SBCL-only, probably not gradual typing), There is not much need to do specialization of generics, or to preserve typing information for next-stage lowering.⁴¹

Because of this, we *did not* actually need a proper system F IR, despite we did, as typeclass elaboration is perfectly doable without it. This is where the author has displayed both his long-term thinking ability and his shortsightedness:

- The former, as in for later proper implementation of arbitrary rank polymorphism and/or inductive families, a System F IR is useful to have;
- The latter, as in after Lambda IR, typing information is gone since it does not preserve types, leaving codegen harder, and most importantly, *much less robust*. Where we have to record the *shapes* of symbols⁴² and to do some form of pattern matching on IR to temporarily fix a bug. Moreover, it is much less feasible if we were to develop some other backends that need types (e.g. C).

Although this does not have much to do with the usefulness of SysF IR, the insight is the same:

It is crucial to have a properly typed IR throughout compilation of programs.

3.1 The *Lambda* IR, Codegen & FFI

Elaboration to System F IR marks the end of “going up”, whereas *Lambda* IR⁴³ serves as an actual intermediate language that marks the beginning of code lowering. *Lambda* IR should be classified as in A-normal form, or ANF for short, a type of IR frequently used in functional programming language compilers.

The essence of ANF is *names*. Object, or any (nontrivial) expression, is referred by name, by immediately capture them in a let-bound variable. This, to some form, mimics the assembly code of the same program. Although *Tigris* compiles to a high-level language, its form is not what normal programming styles looks like and thus SBCL may benefit from it, as the *Tigris* compiler is doing some of the jobs for it.

Definition of *Lambda* IR Design of *Lambda* IR focuses on five aspects: Value, Rhs, Stmt, Tail and LExpr. We shall present the definition of these parts individually, together with a brief explanation of their intents. Readers shall note that these definitions are mutually recursive.

⁴¹though it has been painfully proven to be convenient, through coding practice.

⁴²which itself, is not perfectly reliable.

⁴³The name *Lambda* is a direct reference to the IR Appel used in his *Compiling with Continuations* and presumably used in the SML/NJ compiler. *Tigris* itself is a nod to Appel’s *Modern Compiler Implementation in ML*, of which the cover features a tiger. *Tigris* is latin.

Value Also called immediate values. They cannot be further reduced.

```
inductive Value where
| var    (x : Name)
| cst    (k : Const)
| constr (tag : Name) (fields : Array Name)
| lam    (param : Name) (body : LExpr) -- single-argument lambdas; multi-arg via tuples
deriving Repr, Inhabited
```

Note that *closures* (lambdas) at this stage are not yet converted and capture free variables.

Rhs denotes the right-hand sides of ANF where all arguments are variables

```
inductive Rhs where -- x : Rhs
| prim    (op : PrimOp) (args : Array Name) -- x := op(args1, ..., argsn)
| proj    (src : Name) (idx : Nat)          -- x := srcidx
| mkPair   (a b : Name)                     -- x := ⟨a, b⟩
| mkConstr (tag : Name) (fields : Array Name) -- x := tag[fields1, ..., fieldsn]
| isConstr (src : Name) (tag : Name) (arity : Nat) -- x := src matches ⟨tag/arity⟩
| call     (f : Name) (arg : Name)          -- x := f(arg)
deriving Repr, Inhabited
```

Stmt right now only has single let binding, which, in practice, is no difference from a letexp.

```
inductive Stmt where
| let1 (x : Name) (rhs : Rhs)
deriving Repr, Inhabited
```

Tail denotes the tail positions, which includes return, application, conditional and pattern matching branches. Tail is important in marking tail positions, which is utilized in later tailcall transformations.

```
inductive Tail where -- tl : Tail
| ret    (x : Name) -- tl := RET x
| app    (f : Name) (arg : Name) -- tl := f(arg)T
| cond   (condVar : Name) (tBranch eBranch : LExpr) -- tl := if cond then t else e
| switchConst (scrut : Name) -- tl := caseC of casesi
    (cases : Array (Const × LExpr))
    (default? : Option LExpr)
| switchCtor (scrut : Name) -- tl := case of casesi
    (cases : Array (Name × Nat × LExpr))
    (default? : Option LExpr)
| matchFail (pats : Array Name) -- tl := MATCHFAIL(pats)
deriving Repr, Inhabited
```

Note that constant switches and constructor switches are treated differently – This is to facilitate later constant folding more convenient to implement.

MATCHFAIL is made a special token to denote the *fallback* branch of a partial match. As said before, entering such branch results in a nonrecoverable runtime error. The reason that it takes *pats*, an array of Names i.e. strings, is to provide the users with the symbols of the match discriminants, which, is directly passed onto codegen, printing from the Lisp debugger.

LExpr is the main body of an program in *Lambda IR*.

```

inductive LExpr where
  /-- always ends in tails -/
  | seq    (binds : Array Stmt) (tail : Tail)
  | letVal (x : Name) (v : Value) (body : LExpr) -- let  $x = v$  in body
  /-- Nested let Rhs bindings. -/
  | letRhs (x : Name) (rhs : Rhs) (body : LExpr) -- let  $x = rhs$  in body
  /-- Recursive function bindings. -/
  | letRec (funs : Array LFun) (body : LExpr)
    -- let $\omega$ 
    -- label  $fun_1 : \dots$ 
    -- label  $fun_2 : \dots$ 
    --  $\vdots$ 
    -- in body

```

deriving Repr, Inhabited, BEq

```

/-- A top-level function in the Lambda IR. -/
structure LFun where
  fid    : Name
  param  : Name
  body   : LExpr
deriving Repr, Inhabited

```

Here, normally what would be one `letexp` is divided into 3 sub-lets based on their RHS: a **let** ι that binds immediate value, a **let** that binds regular Rhs, and finally a **let** ω that binds a group of mutually recursive functions.

In addition to these mutually recursive definitions, we additionally have `LFun` that represents a top-level function. Readers with a keen heart will notice that `letRec` carries an array of `LFuns` – `LFun` at this position doesn't mean that `funs` have to be toplevel. We use `LFun` instead of `Name × Name × LExpr` is purely out of performance reasons: they have compatible representation, which, could save some potentially unnecessary conversions.

Sequence operations – `seq` – although *Tigris* is a pure language, we use `seq` to capture tails as it is the only constructor in `LExpr` that takes `Tails`. Also, we may someday turn *Tigris* into a impure functional language like OCaml which, can be convenient if `seq` is present in `LExpr`.

4 Conclusions and Future Work

From our experiments we can conclude that ...